

▼ PDS Practical-1

name: Jafar A.Razzak birbal

DIV: SYA

Roll No: 72

Aim: To study and implement sequence data types in Python.

Theory:

In Python, sequence data types are collections of elements that are stored in a specific order and can be accessed using indexing. The most commonly used sequence types are lists, tuples, strings, and ranges. These data types allow programmers to store multiple values in a single variable and perform various operations on them efficiently.

```
# a. Strings
print("----- STRING -----")
s = "Hello Python"
print("Original:", s)
print("Upper:", s.upper())
print("Slice (0:5):", s[0:5])
print()
# b. Lists
print("----- LIST -----")
lst = [10, 20, 30]
print("Original:", lst)
lst.append(40)
print("After append:", lst)
lst.remove(20)
print("After remove:", lst)
print()
# c. Arrays (using array module) import array
print("----- ARRAY -----")
import array
arr = array.array('i', [1, 2, 3, 4])
print("Original:", arr)
arr.append(5)
print("After append:", arr)
print("Access index 2:", arr[2])
print()
# d. Tuples
print("----- TUPLE -----")
tup = (1, 2, 3, 4)
print("Tuple:", tup)
print("Index 1:", tup[1])
print("Slice:", tup[1:3])
print()
# e. Dictionary
print("----- DICTIONARY -----")
d = {"name": "Pawan", "age": 20}
print("Original:", d)
d["city"] = "Mumbai"
print("After adding:", d)
print("Access name:", d["name"])
print()
# f. Sets
print("----- SET -----")
st = {1, 2, 3}
print("Original:", st)
st.add(4)
print("After add:", st)
st.remove(2)
print("After remove:", st)
print()
# g. Range
print("----- RANGE -----")
r = range(1, 6)
print("Range:", list(r))
for i in r:
    print(i, end=" ")
print()
```

```
----- STRING -----
Original: Hello Python
Upper: HELLO PYTHON
Slice (0:5): Hello
```

```

---- LIST ----
Original: [10, 20, 30]
After append: [10, 20, 30, 40]
After remove: [10, 30, 40]

---- ARRAY ----
Original: array('i', [1, 2, 3, 4])
After append: array('i', [1, 2, 3, 4, 5])
Access index 2: 3

---- TUPLE ----
Tuple: (1, 2, 3, 4)
Index 1: 2
Slice: (2, 3)

---- DICTIONARY ----
Original: {'name': 'Pawan', 'age': 20}
After adding: {'name': 'Pawan', 'age': 20, 'city': 'Mumbai'}
Access name: Pawan

---- SET ----
Original: {1, 2, 3}
After add: {1, 2, 3, 4}
After remove: {1, 3, 4}

---- RANGE ----
Range: [1, 2, 3, 4, 5]
1 2 3 4 5

```

Q) What are sequence data types in Python? Sequence data types in Python are data structures that store multiple elements in an ordered manner. These elements can be accessed using indexing and slicing.

Types of sequence data types:

- Strings
- Lists
- Tuples
- Range

Indexing & Slicing

- Indexing: Accessing elements using position (index starts from 0)
- Slicing: Accessing a group of elements using start:end

✓ Q 2) Difference between List and Tuple

Lists and **Tuples** are both sequence data types in Python used to store collections of items. However, they have key differences:

- **Mutability:** This is the primary difference.
 - **Lists are mutable:** This means their elements can be changed (added, removed, modified) after the list has been created. They are defined using square brackets `[]`.
 - **Tuples are immutable:** This means once a tuple is created, its elements cannot be changed. They are defined using parentheses `()`.
- **Syntax:**
 - **List:** `my_list = [1, 'hello', 3.14]`
 - **Tuple:** `my_tuple = (1, 'hello', 3.14)`
- **Performance:** Tuples are generally faster than lists for iteration and access, especially for large collections, because their size is fixed.
- **Usage:**
 - **Lists** are typically used for collections of items that might change, like a list of user inputs or items in a shopping cart.
 - **Tuples** are often used for heterogeneous collections of data where the elements have a specific meaning (e.g., coordinates (x, y), a record of an event), or when you need to ensure that the data remains constant.

Here's a quick comparison:

Feature	List	Tuple
Mutability	Mutable (changeable)	Immutable (unchangeable)
Syntax	<code>[]</code> (square brackets)	<code>()</code> (parentheses)
Performance	Slower	Faster
Use Case	Collections that change	Fixed collections, records

Double-click (or enter) to edit

Q 2) Difference between List and Tuple

Q3) How are Python arrays different from lists?

When should you use the `array` module instead of a list?

Lists vs. Arrays (from `array` module)

Feature	Python Lists	<code>array</code> Module Arrays
Data Type	Can store different data types (heterogeneous)	Stores only one data type (homogeneous)
Flexibility	More flexible, can grow/shrink dynamically	Less flexible, fixed type for all elements
Memory Usage	More memory-intensive	More memory-efficient for large collections
Performance	Slower for numerical operations	Faster for numerical operations

Q4. Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples? How Dictionaries Work Internally Python dictionaries are implemented as hash tables. Here's a simplified breakdown:

Key-Value Pairs: A dictionary stores data as key-value pairs. Each key must be unique and immutable (like strings, numbers, or tuples).

Hash Function: When you add a key-value pair to a dictionary, Python calculates a hash value for the key using a hash function. This hash value is an integer.

Buckets/Slots: The hash table is divided into a number of 'buckets' or 'slots'. The hash value determines which bucket the key-value pair will be stored in.

Collision Handling: It's possible for different keys to produce the same hash value (a 'hash collision'). Python handles this by using various techniques (like open addressing or chaining) to store multiple items in the same bucket, typically by storing them in a linked list or similar structure within that bucket.

Why Dictionary Lookups are Faster Dictionary lookups are significantly faster than lists or tuples primarily because of the hashing mechanism:

Q5). What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Q5) What is the difference between sets and lists in Python?

How do sets handle duplicate values, and where are sets useful in practical scenarios?

Difference between Sets and Lists:

Feature	Lists	Sets
Order	Ordered (elements maintain insertion order)	Unordered (elements have no specific order)
Mutability	Mutable (elements can be changed)	Mutable (elements can be added/removed, but individual elements are immutable)
Duplicates	Allows duplicate elements	Does not allow duplicate elements
Indexing	Supports indexing and slicing	Does not support indexing or slicing
Syntax	Defined with square brackets <code>[]</code>	Defined with curly braces <code>{}</code> or <code>set()</code>

How Sets Handle Duplicate Values: Sets inherently do not allow duplicate values. When you add an element that already exists in a set, the set simply ignores the addition, and its content remains unchanged. This property makes sets ideal for tasks requiring unique collections of items.

Where Sets Are Useful in Practical Scenarios:

1. **Removing Duplicates:** The most common use case. If you have a list with duplicate items and you want only the unique items, converting it to a set automatically removes all duplicates.

```
my_list = [1, 2, 2, 3, 4, 4, 5]
unique_elements = set(my_list) # {1, 2, 3, 4, 5}
```

2. **Membership Testing:** Checking if an element is present in a large collection is generally faster with sets than with lists (average $O(1)$ for sets vs. $O(n)$ for lists).

```
my_set = {1, 2, 3, 4, 5}
if 3 in my_set: # Very fast
    print('3 is in the set')
```

3. **Mathematical Set Operations:** Sets are optimized for operations like union, intersection, difference, and symmetric difference.

- **Union:** All unique elements from both sets (`set1 | set2`)
- **Intersection:** Common elements in both sets (`set1 & set2`)
- **Difference:** Elements in `set1` but not in `set2` (`set1 - set2`)

- **Symmetric Difference:** Elements in either set, but not in both ($\text{set1} \wedge \text{set2}$)

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}
print(set_a.union(set_b))      # {1, 2, 3, 4, 5, 6}
print(set_a.intersection(set_b)) # {3, 4}
print(set_a.difference(set_b))  # {1, 2}
```

Conclusion

Sequence data types in Python play a crucial role in organizing and managing data efficiently. They provide flexibility through various operations like indexing, slicing, and iteration, making data handling simple and effective. Understanding the differences between mutable and immutable sequences helps in choosing the appropriate data type for specific tasks. Overall, sequence data types enhance coding efficiency and are fundamental to Python programming.